

C Masked dLLM Formulation

Masked diffusion language model sequence of tokens $x_t, t \in [0, 1)$, which follow a forward diffusion process q . This process takes as input the complete sequence x_0 at $t = 0$ and gradually corrupts it by randomly replacing tokens with a mask token `mask`. Therefore, x_t represents the sequence with increasing masking ratios in expectation. Each token in the sequence x_t^i thus follows the conditional distribution,

$$q_{t|0}(x_t|x_0) = \prod_{i=0}^L q_{t|0}(x_t^i|x_0^i), \quad q_{t|0}(x_t^i|x_0^i) = \begin{cases} 1 - \alpha_t, & x_t^i = \text{mask} \\ \alpha_t, & x_t^i = x_0^i \end{cases} \quad (5)$$

where α_t (a.k.a noise schedule) is strictly decreasing in t . Simply put, at any timestep, the probability that a token transitions to the masked state is α_t . At the end of the forward process, i.e. at $t = 1$, all tokens are guaranteed to be masked.

This masked sequence serves as the input for the reverse process. A key property of the forward process is that once a token transitions to the masked state, it cannot transition to any other state. Therefore, the conditional distribution from an arbitrary time step t to s (i.e., the reverse process), such that $0 \leq s < t \leq 1$ is given by,

$$q_{s|t}(x_s^i|x_t) = \begin{cases} 1, & x_t^i \neq \text{mask}, x_s^i = x_t^i \\ \frac{1-\alpha_s}{1-\alpha_t}, & x_t^i = \text{mask}, x_s^i = \text{mask} \\ \frac{\alpha_s-\alpha_t}{1-\alpha_t} q_{0|t}(x_s^i|x_t), & x_t^i = \text{mask}, x_s^i \neq \text{mask} \\ 0, & \text{otherwise} \end{cases} \quad (6)$$

The function $q_{0|t}(x_s^i|x_t)$ is estimated by the language model, that predicts the original token in sequence x_0 , if it is masked in x_t . Notably, previous works find that the model does not require the timestep as an input [] since the number of mask tokens implicitly provide this information to the model.

The model, parameterized as $f_\theta(\cdot|x_t)$ learns to predict all the masked tokens in the sequence x_t simultaneously, similar to the masked language modeling task. More specifically, it is trained by minimizing a NELBO of the negative log-likelihood, given by,

$$\text{NELBO}(\theta) \triangleq \mathbb{E}_{x_0, x_t} \left[\int_{t=0}^{t=1} \frac{\alpha'_t}{1-\alpha_t} \sum_{i=1}^L \mathbb{1}[x_t^i = \text{mask}] \log f_\theta(x_0^i | x_t) \right], \quad (7)$$

where x_0 is sampled from the training data distribution p_{data} , and $x_t \sim q_{t|0}(\cdot|x_0)$. In summary, the model is trained to reverse the forward process by gradually denoising (unmasking) the input sequence (all masked tokens) and recover the data distribution.

While various forms of noise schedules can be used [36, 39], Nie et al. [30, LLaDA] uses the linear schedule: $\alpha_t = 1 - t$. The resulting loss function is a specific form of Equation (7):

$$-\mathbb{E}_{t \sim \mathcal{U}[0,1], x_0, x_t} \left[\frac{1}{t} \sum_{i=1}^L \mathbb{1}[x_t^i = \text{mask}] \log f_\theta(x_0^i | x_t) \right]. \quad (8)$$

D Experiment Details

Inference To decode a sequence of N tokens, we use $\frac{N}{2}$ denoising steps and unmask 2 tokens in each step. While the decoding process can generate tokens in any order, we find that decoding from left to right in blocks yields slightly better performance in practice. This is referred to as the semi-autoregressive decoding strategy [30]. More specifically, we divide the sequence into blocks of 32 tokens. In each step, we unmask 2 tokens with the highest confidence within the current block, regardless of their position. Once all the tokens in the current block are unmasked, we move to the next one.

D.1 *diffu*-GRPO

We use the TRL library [43] to implement *diffu*-GRPO. For our *diffu*-GRPO training, we employed Low-Rank Adaptation (LoRA) with a rank of $r = 128$ and scaling factor $\alpha = 64$.

For *diffu*-GRPO on gsm8k, math, countdown and sudoku tasks, training was conducted on 8 NVIDIA A100-80G GPUs, with the following hyperparameters: sequence length of 256 tokens, batch size of 6 per GPU, and gradient accumulation steps of 2. We optimized the model using the AdamW optimizer [25], with parameters $\beta_1 = 0.9$, $\beta_2 = 0.99$, weight decay of 0.1, learning rate of 3×10^{-6} , and gradient clipping at 0.2. For computational efficiency, we utilized Flash Attention 2 [11] and 4-bit quantization. In gradient update iterations, each token in the prompt is randomly masked with a probability $p_{\text{mask}} = 0.15$ for log-probability estimation. Our codebase contains further configuration details: <https://github.com/dllm-reasoning/d1>. We train 7700, 6600 steps (number of gradient updates) for GSM8K and MATH500 respectively; for Countdown and Sudoku, we train on synthetic generated datasets for 5000, 3800 steps respectively.

For *diffu*-GRPO on coding task, training was conducted on 4 NVIDIA RTX A5000 for 7500 steps (base model + *diffu*-GRPO) and 9000 steps (SFT model + *diffu*-GRPO), with a per-device batch size of 2 and 4 gradient accumulation steps. The other hyperparameters remain the same as other tasks. Exact configuration details have been provided in our codebase.

D.1.1 Reward Functions, RL Training, and Evaluation Datasets

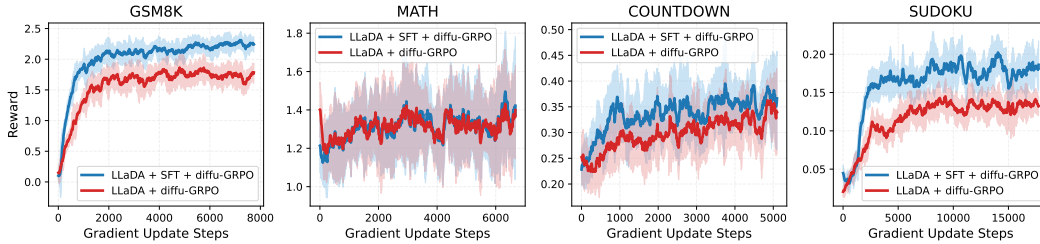


Figure 7: Reward curves during RL training for the models in Table 1, across four reasoning tasks. We compare LLaDA+*diffu*-GRPO and *d1*-LLaDA (+SFT + *diffu*-GRPO). *d1*-LLaDA consistently achieves higher or comparable reward trajectories.

We designed specific reward functions to guide the model’s learning for each task. The rewards are structured to encourage proper formatting, accurate reasoning, and correct solutions, with varying levels of granularity depending on task requirements. We show the training curves of the results in Table 1 in Figure 7.

GSM8K For the GSM8K dataset, we conduct RL on the training split of the GSM8K dataset⁶ and evaluate on the test split. We employ a composite reward function consisting of five components following the unsloth implementation of reward functions⁷, we used these:

- **XML Structure Reward:** Rewards proper formatting with reasoning and answer tags:
– +0.125 for each correctly placed opening and closing tag

⁶<https://huggingface.co/datasets/openai/gsm8k>

⁷<https://unsloth.ai/blog/r1-reasoning>

- Small penalties for extraneous content after closing tags
- **Soft Format Reward:** Awards 0.5 points for responses matching the pattern:
`<reasoning>...(content)...</reasoning><answer>...(content)...</answer>`
- **Strict Format Reward:** Awards 0.5 points for adhering to the exact prescribed format with appropriate line breaks.
- **Integer Answer Reward:** Awards 0.5 points if the extracted answer is a valid integer.
- **Correctness Reward:** Awards 2.0 points if the extracted answer exactly matches the ground truth.

Countdown For the Countdown task, we train on the training split of the dataset⁸ from the TinyZero project [34], restricting to instances that use only three numbers. And we evaluate on 256 synthetically generated countdown questions with 3 numbers. We implement a reward function that checks if an arithmetic expression constructed from given numbers reaches a target value:

The function awards:

- 1.0 point when the equation equals the target and uses exactly the available numbers
- 0.1 points when the equation uses the right numbers but doesn’t reach the target
- 0 points otherwise

Sudoku For the 4×4 Sudoku task, we utilize the training dataset available at <https://github.com/Black-Phoenix/4x4-Sudoku-Dataset>, specifically the subset containing one million unique puzzles. This dataset was synthetically generated using code from Arel [3]. For evaluation purposes, we randomly generate 256 Sudoku puzzles using this generator. The reward is calculated as the proportion of correctly filled cells among those that were empty in the original puzzle. This approach focuses evaluation on the model’s problem-solving ability rather than its capacity to copy pre-filled values.

MATH500 For the MATH500 task, we train on the train split of the MATH dataset⁹. Like GSM8k, we employ a composite reward function consisting of:

- **Format Reward:** We award format reward points depending on the presence of `<answer></answer>` tags and `\boxed`, as follows:
 - 1.00 point if answer tags are present with `\boxed` inside them
 - 0.75 points if answer tags are present without `\boxed` in them
 - 0.50 points if answer tags are not present, but `\boxed` is present
 - 0.25 points if neither answer tags, nor `\boxed` is present
- **Correctness Reward:** 2.0 points if the correct answer is in `\boxed{}`

Coding For the coding model, we train on the KodCode-Light-RL-10k¹⁰ dataset. Again, we use a composite reward function comprising of:

- **XML Structure Reward:** The same function used for GSM8k is also used for this task, with the addition that an extra 0.5 points are provided if the program is within answer tags. Additionally, 0 points are awarded if the code is not wrapped in `‘‘python‘‘`.
- **Correctness Score:** Similar to [14, 27], we use unit tests to verify the correctness of the code. Notably, while these works use a binary reward, we use the fraction of unit tests passed as the reward.
- **Safe Code:** To prevent the generation of unsafe code, we assign a reward of 0 if any blocked modules are used. These include `os`, `sys`, `shutil`, `subprocess`, `socket`, `psutil`, `ctypes`, `pathlib`, `builtins`, and `__import__`.

⁸<https://huggingface.co/datasets/Jiayi-Pan/Countdown-Tasks-3to4>

⁹<https://huggingface.co/datasets/ankner/math-500>

¹⁰<https://huggingface.co/datasets/KodCode/KodCode-Light-RL-10K>